Algorithms for triple-word arithmetic

Nicolas Fabiano and Jean-Michel Muller

Abstract—Triple-word arithmetic consists in representing high-precision numbers as the unevaluated sum of three floating-point numbers. We introduce and analyze various algorithms for manipulating triple-word numbers. Our algorithms are faster and more accurate than what one would obtain by just using the usual floating-point expansion algorithms in the special case of expansions of length 3. A RETIRER.

1 Introduction and notation

Numerical calculations sometimes require a precision significantly higher than the one offered by the basic floating-point formats. This occurs for instance when evaluating transcendental functions with correct rounding: it is almost impossible to guarantee last-bit accuracy in the final result if all intermediate calculations are done in the target precision. For instance, the CRLibm library [**?**] of correctly rounded elementary functions uses "double-double" or "triple-double" [**?**], [**?**] operations in the last steps of the evaluation of approximating polynomials.

Double-double arithmetic had been useful for implementing BLAS [**?**]. Bailey, Barrio and Borwein [**?**] give several timely examples in mathematical physics and dynamics where higher precisions are needed.

Arbitrary precision libraries such as GNU-MPFR [**?**] have the advantage of being versatile, but may involve a significant penalty in terms of speed and memory consumption if one only requires calculations accurate within around 150 bits in a few critical parts of a numerical program.

Double-word and Triple-word arithmetics consist in representing a real number as the unevaluated sum of two and three floating-point numbers, respectively. They are frequently called "double double" and "triple double", because in practice the underlying format being used is the binary64/double precision format of the IEEE 754 Standard on Floating-Point Arithmetic [**?**], [**?**].

A generalization of these arithmetics is the notion of floating-point expansion [**?**], [**?**], [**?**], where a high-precision number is represented as the sum of n floating-point numbers.

Algorithms for double word arithmetic have been presented in [**?**], [**?**]. The purpose of this paper is to introduce and analyze efficient algorithms for performing the arithmetic operations in triple word arithmetic. Our goal is to obtain algorithms that are faster and error bounds that are smaller than the ones we could obtain simply by using floating-point expansion algorithms in the particular case $n = 3$.

In the following, we assume a radix-2, precision-p floating-point (FP) arithmetic system, with unlimited exponent range and correct rounding. As a consequence, our results will apply to "real-world" binary floating-point arithmetic, such as the one specified by the IEEE 754-2008 Standard provided that underflow and overflow do not occur.

The notation $\mathrm{RN}(t)$ stands for t rounded to the nearest FP number, ties-to-even. The number $\mathrm{ulp}(x)$, for $x \neq 0$ is $2^{\lfloor \log_2 |x| \rfloor - p + 1}$, and $u = 2^{-p} = \frac{1}{2}\mathrm{ulp}(1)$ denotes the roundoff error unit. When an arithmetic operation $a \nabla b$ is performed, where a and b are FP numbers, what is effectively computed is $\mathrm{RN}(a \nabla b)$, and if t is a real number and $T = \mathrm{RN}(t)$, then

$$|t - T| \leq \frac{u}{1+u} \cdot |t| \leq u \cdot |t|.$$

The algorithms presented in this paper (as well as the usual algorithms that manipulate double words or FP expansions) use as basic blocks Algorithms 1, 2, and 3 below.

Algorithm 1 – Fast2Sum(a, b). The Fast2Sum algorithm [**?**].

$s \leftarrow \mathrm{RN}(a + b)$
$z \leftarrow \mathrm{RN}(s - a)$
$t \leftarrow \mathrm{RN}(b - z)$

If $|a| \geq |b|$ then values s and t computed by Algorithm 1 satisfy $s+t = a+b$. Hence, t is the error of the FP addition $s \leftarrow \mathrm{RN}(a + b)$.

Algorithm 2 – 2Sum(a, b). The 2Sum algorithm [**?**], [**?**].

$s \leftarrow \mathrm{RN}(a + b)$
$a' \leftarrow \mathrm{RN}(s - b)$
$b' \leftarrow \mathrm{RN}(s - a')$
$\delta_a \leftarrow \mathrm{RN}(a - a')$
$\delta_b \leftarrow \mathrm{RN}(b - b')$
$t \leftarrow \mathrm{RN}(\delta_a + \delta_b)$

- *N. Fabiano is with École Normale Supérieure, 45 Rue d'Ulm, 75005 Paris, France. E-mail: nicolabiano22@yahoo.fr*
- *J.-M. Muller is with CNRS, Laboratoire LIP, ENS Lyon, INRIA, Université Claude Bernard Lyon 1, Lyon, France. E-mail: jean-michel.muller@ens-lyon.fr*

Algorithm 2 requires twice as many operations as Algorithm 1, but its output variables always satisfy $s + t = a + b$: no test on the respective magnitudes of $|a|$ and $|b|$ is needed.

Algorithm 3 – Fast2Mult(a, b)**.** The Fast2Mult algorithm (see for instance [**?**], [**?**], [**?**]). It requires the availability of a fused multiply-add (FMA) instruction for computing RN$(ab - \pi)$.

$\pi \leftarrow \text{RN}(a \cdot b)$
$\rho \leftarrow \text{RN}(a \cdot b - \pi)$

The values π and ρ computed by Algorithm 3 satisfy $\pi + \rho = ab$. Algorithm 3 requires the availability of an FMA (*fused multiply-add*) instruction, that evaluates expressions of the form $ab + c$ with one final rounding only.

[EN VRAC]

Definition 1. *A sequence is said F-nonoverlapping (with Fabiano's defintion, introduced in this paper) when* $\forall i, |x_{i+1}| \leq \frac{1}{2}\text{uls}(x_i)$

Definition 2. *A sequence is said P-nonoverlapping (with Priest's defintion, [REF]) when* $\forall i, |x_{i+1}| < \text{ulp}(x_i)$

Definition 3. *For any definition of nonoverlapping, a sequence is said nonoverlapping wIZ (with interleaving zeros) when we have a set* I_0 *such that* $\forall i \in I_0, x_i = 0$*, and* $(x_i)_{i \notin I_0}$ *nonoverlapping.*

Definition 4. *We call Double Word (DW) a pair* (x_0, x_1) *of FP such that* $x_0 = RN(x_0 + x_1)$*.*

Definition 5. *We call Triple Word (TW) a triplet* (x_0, x_1, x_2) *of FP that is P-nonoverlapping.*

The rest of the paper is organized as follows. Section 2 presents the classical basic blocks that will be used in the rest of the paper, and some origninal theorems related to them. Section 3 proves that the classical algorithm to turn a arbitrary sequence into an expansion works for TW. Section 4 presents an algorithm to correctly round a TW, and proves it correctness. Section 5 does the same as section 3 for the sum of two TW. Section 6 presents two versions of an original algorithm for the product of two TW, and proves their correctness and tight error bounds. Section 7 does the same for the product of a DW by a TW. Section 8 does the same for the reciprocal of a TW. Section 9 does the same for the quotient of two TW. Section 10 [???].

2 Basic blocks

Algorithms on TW will use classical basic blocks than can be proven once for all to work properly.

2.1 2Sum and Fast2Sum

The sum of two FP can be computed into a DW using algorithm 4 for 6 flops. If we know that there are some exponents $k_a \geq k_b$, then we can use algorithm 5 for only 3 flops instead.

Algorithm 4 – 2Sum(a, b). (6 flops) [**?**], [**?**].

Ensure: $(s, e) = a + b$
$s \leftarrow \text{RN}(a + b)$
$a' \leftarrow \text{RN}(s - b)$
$b' \leftarrow \text{RN}(s - a')$
$\delta_a \leftarrow \text{RN}(a - a')$
$\delta_b \leftarrow \text{RN}(b - b')$
$e \leftarrow \text{RN}(\delta_a + \delta_b)$

Algorithm 5 – Fast2Sum(a, b). (3 flops) [**?**].

Require: $\exists k_a \geq k_b$
Ensure: $(s, e) = a + b$
$s \leftarrow \text{RN}(a + b)$
$z \leftarrow \text{RN}(s - a)$
$e \leftarrow \text{RN}(b - z)$

Sometimes, we know in advance the value of s. In that case, e can be computed saving the first operation, with algorithms denoted by (Fast)2Sum$_2(s)(x, y)$.

2.2 2Prod

[PARLER DE LA FMA EN INTRO]

Assuming that a fma is available, the product of two FP can be computed into a DW using algorithm 6 for 2 flops.

Algorithm 6 – 2Prod(a, b)**.** (2 flops) [**?**], [**?**], [**?**])

Ensure: $(\pi, e) = a \cdot b$
$\pi \leftarrow \text{RN}(a \cdot b)$
$e \leftarrow \text{RN}(a \cdot b - \pi)$

Remark 1. *When no fma is available, this operation is much harder: the best known algorithm uses 17 flops.*

2.3 VecSum

The aim of this algorithm is to turn a sequence that is slightly nonoverlapping into one that is more nonoverlapping, with no error.

Algorithm 7 – VecSum$(x_0, \ldots, x_{n-1})$**.** ($6n-6$ flops) [REF]

Ensure: $e_0 + \cdots + e_{n-1} = x_0 + \cdots + x_{n-1}$
$s_{n-1} \leftarrow x_{n-1}$
for $i = n - 2$ **to** 0 **do**
  $s_i, e_{i+1} \leftarrow \text{2Sum}(x_i, s_{i+1})$
end for
$e_0 \leftarrow s_0$

There are several theorems related to this algorithm, with different definitions of nonoverlapping. In what follows, we will use the original theorem:

Theorem 1. *We suppose, after removing the interleaving zeros, that we can write in a non-necessarily canonical way* $x_i = M_i 2^{k_i - p+1}$, $|M_i| < 2^p$ *such that*

$$\forall i \leq n-2, k_{i-1} \geq k_i + 1$$

$$k_{n-2} \geq k_{n-1}$$

Then VecSum$(x_0, \cdots, x_{n-1})$ *is F-nonoverlapping wIZ with same sum.*

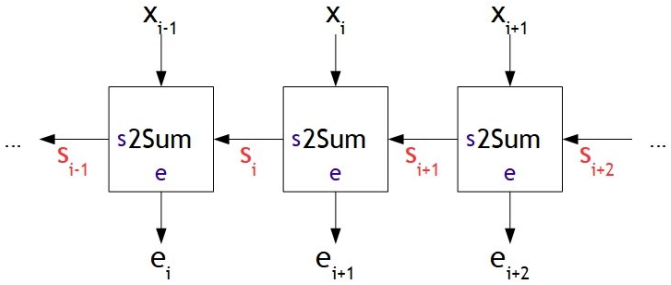
In this case, Fast2Sum can be used, so that it only costs $3n-3$ *flops.*

Proof: We notice that interleaving zeros in the input simply give some in the output without changing the non-zero terms, so that we can suppose that we have removed them.

Firstly, $\forall i, |s_i| \leq (2-2u)2^{k_{i-1}}$.

Indeed, if by induction $|s_{i+1}| \leq (2-2u)2^{k_i}$, given $|x_i| \leq (2-2u)2^{k_i}$ we get $|s_{i+1}| + |x_i| \leq (4-4u)2^{k_i} \leq (2-2u)2^{k_{i-1}}$ so $|s_i| \leq (2-2u)2^{k_{i-1}}$.

This gives $|e_i| \leq 2u2^{k_{i-1}}$, and justifies Fast2Sum being used.



We suppose that $|e_i| > \frac{1}{2}\text{uls}(e_{i'})$ with $i' < i$.

We suppose WLOG that $\text{uls}(e_{i'}) = u$.

We easily get by induction that for all i, if $2^k | s_i, x_{i-1}, \ldots, x_0$, then $2^k | e_i, \ldots, e_0$. Yet $|e_i| \leq \frac{1}{2}\text{ulp}(s_{i-1})$ so $|s_{i-1}| \geq 1$ so it is multiple of $2u$. Given we want a $e_{i'}$ non-multiple of $2u$, that must be the case for one of the $x_j, j \leq i-2$. In particular, we have $2^{k_j} \leq \frac{1}{2}$ so by isotony $2^{k_{i-2}} \leq \frac{1}{2}$ so $2^{k_{i-1}} \leq \frac{1}{4}$, which contradicts $|e_i| \leq 2u2^{k_{i-1}}$. □

The conditions on the input of this theorem are a bit heavy, so we will use the following corollary:

Corollary 2. *We suppose that we have* $I \subset [\![1, n-2]\!]$ *with no 2 consecutive indices such that*

$$\forall i \in [\![0, n-2]\!] \backslash I, \text{ufp}(x_{i+1}) \leq \frac{1}{2}\text{ufp}(x_i)$$

$$\forall i \in I, \text{ufp}(x_{i+1}) \leq 2^{p-2}\text{uls}(x_i) \wedge \text{ufp}(x_{i+1}) \leq \frac{1}{4}\text{ufp}(x_{i-1})$$

Then VecSum$(x_0, \cdots, x_{n-1})$ *is F-nonoverlapping with same sum.*

In this case, Fast2Sum can be used, so that it only costs $3n-3$ *flops.*

Proof: For $i \notin I$, we take $k_i = e_{x_i}$ the canonical exponent.

For $i \in I$, we take $k_i = \max(k_{i+1}+1, e_{x_i})$. This is possible because:

$$2^{k_{i+1}-p+2} | x_i$$

$$2^{e_{x_i}-p+1} | x_i$$

$$\rightarrow 2^{k_i - p+1} | x_i$$

$$|x_i| \leq 2 \cdot 2^{e_{x_i}}$$

$$\rightarrow |x_i| \leq 2 \cdot 2^{k_i}$$

For $i, i+1 \notin I$, we have $k_{i+1} \leq k_i - 1$.

For $i \in I$, we have on one hand $k_{i+1} \leq k_i - 1$, and on the other hand $e_{x_i} \leq k_{i-1} - 1$ and $k_{i+1} \leq k_{i-1} - 2$ so $k_i \leq k_{i-1} - 1$.

This allows us to use Theorem 1. □

2.4 VecSumErrBranch

This algorithm is similar to the previous one, but sums are computed starting with the larger terms, and some branching help maximising the concentration of bits:

Algorithm 8 – VSEB$(e_0, \ldots, e_{n-1})$**.** ($6n-6$ flops & $n-2$ tests) [REF]

Ensure: $y_0 + \cdots + y_{n-1} = e_0 + \cdots + e_{n-1}$
  $j \leftarrow 0$
  $\epsilon_0 \leftarrow e_0$
  for $i = 0$ **to** $n-3$ **do**
    $r_i, \epsilon_{i+1} \leftarrow \text{2Sum}(\epsilon_i + e_{i+1})$
    if $\epsilon_{i+1} \neq 0$ **then**
      $y_j \leftarrow r_i$
      incr j
    else
      $\epsilon_{i+1} \leftarrow r_i$
    end if
  end for
  $y_j, y_{j+1} \leftarrow \text{2Sum}(\epsilon_{n-2} + e_{n-1})$
  $y_{j+2}, \ldots, y_{n-1} \leftarrow 0$

Remark 2. *Instead of computing* ϵ_{i+1} *and testing if it is zero, we could use flags to detect when the operation* $r_i = RN(\epsilon_i + e_{i+1})$ *is inexact, and compute* ϵ_{i+1} *only in that case using 2Sum*$_2$*. If flags are cheap to acceed, this can be faster.*

In what follows, we will use the original theorem:

Theorem 3. *If* (e_i) *is F-nonoverlapping wIZ, then VSEB*$(x_0, \cdots, x_{n-1})$ *is P-nonoverlapping with same sum, provided that* $p \geq n-1$.

In this case, Fast2Sum can be used, so that it only costs $3n-3$ *flops.*

Proof: We notice that interleaving zeros in the input a ignored without changind anything, so that we can suppose that we have removed them.

⋆ We write $e_i = M_i \cdot 2^{k_i}$ with $|M_i| < 2^p$ odd so that $|e_{i+1}| \leq \frac{1}{2}2^{k_i}$.

We have easily by induction that, for all i, r_{i-1} and ϵ_i are multiples of 2^{k_i}.

⋆ Let i_0 be such that $|\epsilon_{i_0}| = 2^{k_{i_0}}$.

Let us show by induction that for all $i_0 \le i \le n-2$, we have $|r_i| \le 2^{k_{i_0}}(2 - 2^{i_0-i-1})$.

- We can initialize with $i = i_0 - 1$, because what is transmitted to next step (playing the role of r_{i_0-1}) is ϵ_{i_0}, which exactly satisfies the condition.

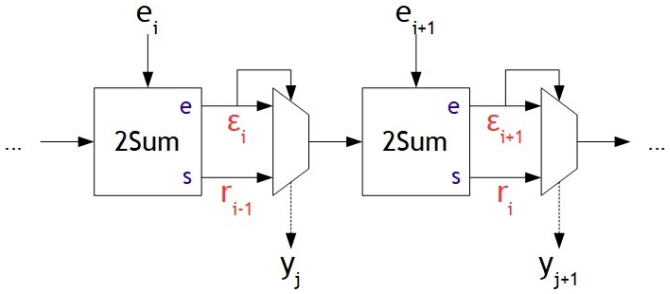
- We suppose that $|r_{i-1}| \le 2^{k_{i_0}}(2 - 2^{i_0-i})$.

We have $|e_{i+1}| \le \frac{1}{2}2^{k_i} \le \frac{1}{4}2^{k_{i-1}} \le \cdots \le 2^{i_0-i-1}2^{k_{i_0}}$.

Thus $|r_{i-1}| + |e_{i+1}| \le 2^{k_{i_0}}(2 - 2^{i_0-i-1})$, and this is a FP because $i_0 \ge 1$ and $i \le n-2$ give $i_0 - i - 1 \ge -n + 2 \ge -p + 1$, so that $|r_i| \le 2^{k_{i_0}}(2 - 2^{i_0-i-1})$.

- This gives the result by induction.

In particular, with j such that $y_j = r_{i_0-1}$, we have $\mathrm{ulp}(y_j) \ge 2|\epsilon_{i_0}| = 2 \cdot 2^{k_{i_0}}$ so $|y_{j+1}| < \mathrm{ulp}(y_j)$.



⋆ For i_0 such that $|\epsilon_{i_0}| > 2^{k_{i_0}}$, we similarly get that for all $i \ge i_0$ such that $\epsilon_{i_0+1}, \cdots, \epsilon_i = 0$, we have $|r_i| \le |\epsilon_{i_0}| + 2^{k_{i_0}}$.

Indeed, the same proof replacing $2^{k_{i_0}}(2 - \cdots)$ by $|\epsilon_{i_0}| + 2^{k_{i_0}}(1 - \cdots)$ works, except that the equality case can be reached in case of errors.

In this case, this is sufficient given $\mathrm{ulp}(y_j) \ge 2|\epsilon_{i_0}| > |\epsilon_{i_0}| + 2^{k_{i_0}} \ge y_{j+1}$.

⋆ Finally, we can use Fast2Sum.

Indeed, we have $2^{k_i} | e_0, \cdots, e_i$ so $2^{k_i} | \epsilon_i$, and $|e_{i+1}| \le 2^{k_i}$. □

Usually, we don't want to keep the complete output, but only a fixed number of terms. The resulting algorithm is denoted by $\mathrm{VSEB}(k)(e_0, \ldots, e_{n-1})$.

It costs as many flops, but the number of tests depends on how this is performed in practice. In what follows, we will assume in our complexity analysis that all is unrolled (which is reasonable because typically $n = 4$), so that only the $n-2$ tests are required.

Theorem 4. *If the total output would be P-nonoverlapping, then the relative error caused by keeping only the first k terms is bounded by $2u^k - 4.2u^{k+1}$, provided that $p \ge 6$.*

Proof: We have by P-nonoverlapping:

$$\mathrm{ufp}(y_k) \le u\,\mathrm{ufp}(y_{k-1}) \le \cdots \le u^k \mathrm{ufp}(y_0)$$

$$|y_k| + \cdots + |y_{n-1}| \le (2-2u)(u^k + u^{k+1} + \cdots + u^n)\mathrm{ufp}(y_0)$$

$$|y_k + \cdots + y_{n-1}| \le 2u^k \mathrm{ufp}(y_0)$$

$$|y_0 + \cdots + y_{n-1}| \ge |y_0| - |y_1 + \cdots + y_{n-1}| \ge (1-2u)\mathrm{ufp}(y_0)$$

$$|y_k + \cdots + y_{n-1}| \le \frac{2u^k}{1-2u}|y_0 + \cdots + y_{n-1}|$$

Thus the relative error is bounded by $2u^k + 4.2u^{k+1}$, provided that $p \ge 5$. □

2.5 Composed algorithm

Of course, there two algorithms are designed to be composed. What is more, in that case, we notice that the first 2Sum in VSEB can be skipped because (e_0, e_1) is already a DW.

When Fast2Sum can be used everywhere, we get a total of $6n - 9$ flops and $n - 2$ tests.

3 Arbitrary to TW

Before manipulating TW, we want to be able to turn any three FP into a TW, with no error. (equivalent of 2Sum)

We can use algorithm 9 [REF]:

Algorithm 9 – ToTW(a, b, c)**.** (21 flops & 1 test)

Ensure: $\bar{r}$ TW **and** $\bar{r} = a + b + c$

$d_0, d_1 \leftarrow \mathrm{2Sum}(a, b)$

$e_0, e_1, e_2 \leftarrow \mathrm{VecSum}(d_0, d_1, c)$

$r_0, r_1, r_2 \leftarrow \mathrm{VSEB}(e_0, e_1, e_2)$

Theorem 5. *If a, b, c FP, then $toTW(a, b, c)$ TW, provided that $p \ge 4$.*

Proof: If (e_0, e_1, e_2) is F-nonoverlapping, then Theorem 3 concludes.

First, $|e_1| \le \frac{1}{2}\mathrm{ulp}(e_0)$ gives (e_0, e_1) F-nonoverlapping.

We denote $s := \mathrm{RN}(d_1 + z)$ the intermediate value in $\mathrm{VecSum}(d_0, d_1, c)$.

- If $e_1 \ne 0$, we suppose WLOG that $\mathrm{uls}(e_1) = u$, and $e_2 > \frac{1}{2}u$.

Then $|e_2| \le \frac{1}{2}\mathrm{ulp}(s)$ gives $s \ge 1$ so $2u | s$ but e_1 is not divisible by $2u$ so d_0 neither so $d_0 < 1$ so $|d_1| \le \frac{1}{2}\mathrm{ulp}(d_0) \le \frac{1}{2}u$.

What is more, $|c + d_1| \ge 1 + \frac{1}{2}u$

Thus $|c| \ge (1 + \frac{1}{2}u) - \frac{1}{2}u = 1$ so $\mathrm{ulp}(c) \ge 2u > 2|d_1|$ so $s = c$ and $e_2 = d_1$, which is excluded because $|e_2| > \frac{1}{2}u \ge |d_1|$.

So (e_1, e_2) is F-nonoverlapping too.

- If $e_1 = 0$, then the same reasonning with e_0 instead of e_1 works. □

Another typical way of forming a TW consists in using any of the following algorithms, but with inputs DW instead of TW (with an implicit third term equal to zero, which is simplified by removing useless operations).

4 TW TO FP

Once our calculations are done, we may want to obtain the FP the closest to our TW result. This is for instance the case when we are implementing correctly rounded elementary functions.

We can use algorithm 10:

Algorithm 10 – FromTW(x_0, x_1, x_2)**.** (6 flops & 1 test)

Require: $\bar{x}$ TW
Ensure: $y = \text{RN}(\bar{x})$
  $a_1, a_2 \leftarrow \text{Fast2Sum}(x_1, x_2)$
  $y \leftarrow \text{RN}(x_0 + a_1)$
  if $\text{RN}(x_0 + 2a_1)$ (fma) exact operation **then**
    if $a_2 > 0$ **then**
      $y \leftarrow \text{RU}(x_0 + a_1)$
    end if
    if $a_2 < 0$ **then**
      $y \leftarrow \text{RD}(x_0 + a_1)$
    end if
  end if

Remark 3. *The fact that the operation is exact can be detected with a flag, but if it is too complicated to acceed we can instead compute Fast2Sum*$(x_0, 2a_1)$ *and test whether the error term is zero, for 3 more flops.*

Remark 4. *Instead of testing* $a_2 > 0$ *(resp.* $a_2 < 0$*), it may be better to test something like* $a_2 > 32u|a_1|$ *(resp.* $a_2 < -32u|a_1|$*) and to raise a warning if none of them is true, because it means that we are in the grey zone due to rounding errors.*

Theorem 6. *If* $\bar{x}$ *TW, then* $FromTW(\bar{x}) = RN(\bar{x})$*.*

Proof: We have $|a_1| \leq \text{ulp}(x_0)$ and $|a_2| \leq \frac{1}{2}\text{ulp}(a_1)$.

Thus, the first test is true iff $x_0 + a_1$ is a FP or is precisely at the middle of two adjacent FP.

- If $x_0 + a_1$ is a FP, then it is easy to check that it is $\text{RN}(\bar{x})$.

- If $x_0 + a_1$ is precisely at the middle of two adjacent FP, then the rounding is simply decided by the sign of a_2.

- In any other case, it is easy to check that $\text{RN}(x_0 + a_1 + a_2) = \text{RN}(x_0 + a_1)$. □

5 SUM

To compute the sum of two TW, we simply use the composition of VecSum and VSEB after a sort of the input:

Algorithm 11 – 3Sum$(x_0, x_1, x_2, y_0, y_1, y_2)$**.** (42 flops & 8 tests)

Require: $\bar{x}$ and $\bar{y}$ TW
Ensure: $\bar{r}$ TW **and** $\left|\frac{\bar{r}-(\bar{x}+\bar{y})}{\bar{x}+\bar{y}}\right| \leq 2u^3 + 4.2u^4$
  $z_0, \ldots, z_5 \leftarrow Merge((x_0, x_1, x_2), (y_0, y_1, y_2))$
  $e_0, \ldots, e_5 \leftarrow \text{VecSum}(z_0, \ldots, z_5)$
  $r_0, r_1, r_2 \leftarrow \text{VSEB(3)}(e_0, \ldots, e_5)$

Remark 5. *This algorithm would also work to compute the sum of a DW and a TW, or of two DW, with same correctness theorem and same error bound but with less flops and tests.*

5.1 Correctness

Theorem 7. *Let* $x_0, \cdots, x_5$ *be FP such that*

$$\forall i, |x_{i+1}| \leq |x_i|$$

$$\forall i, |x_{i+2}| < \text{ulp}(x_i)$$

Then VSEB $\circ$ *VecSum*$(x_0, \cdots, x_5)$ *is P-nonoverlapping, provided that* $p \geq 4$*.*

Remark 6. *This theorem is very special for at most 6 inputs. Indeed, for 7, we can consider*

$$(x_i) = (1-u, -1+2u, -u+u^2, u-u^2, u^2-u^3, u^2-u^3, u^3-u^4)$$

which gives

$$(e_i) = (u, u^2, u^2, -u^3, -u^4)$$

and finally

$$(y_i) = (u, 2u^2, -u^3, -u^4)$$

with $2u^2 = \text{ulp}(u)$*.*

This is essentially why this is reasonable to use P-nonoverlapping for TW but not for general expansions, for which this algorithm preservs only ulp*-nonoverlapping. [REF]*

Proof: $\star$ First, let us prove by descending induction that $|s_i| \leq 2\text{ufp}(x_{i-1})$ and $|s_i| \leq 4\text{ufp}(x_i)$.

- The initialization is clear.

- We have $|s_{i+1}| \leq 4\text{ufp}(x_{i+1}) \leq 4u\text{ufp}(x_{i-1})$ and $|x_i| \leq (2-2u)\text{ufp}(x_{i-1})$ so $|s_{i+1}| + |x_i| \leq (2+2u)\text{ufp}(x_{i-1})$ so after rounding (ties-to-even) $|s_i| \leq 2\text{ufp}(x_{i-1})$.

- We have $|s_{i+1}| \leq 2\text{ufp}(x_i)$ and $|x_i| \leq 2\text{ufp}(x_i)$ so $|s_{i+1}| + |x_i| \leq 4\text{ufp}(x_i)$ so $|s_i| \leq 4\text{ufp}(x_i)$.

This gives the result by induction.

What is more, we still have $|e_i| \leq 2u\text{ufp}(x_{i-1})$.

$\star$ Let us suppose that $\text{uls}(e_j) = u$ and $e_i \geq \frac{5}{8}u$ for some $j < i$.

We are trying to find some conditions that will be used in the next steps.

$|e_i| \leq \frac{1}{2}\text{ulp}(s_{i-1})$ gives $|s_{i-1}| \geq 1$.

Since $2u$ divides s_{i-1} but not e_j, $\exists i' \leq i-2, \neg(2u | x_{i'})$ so $|x_{i'}| < 1$ so by isotony $|x_{i-2}| < 1$.

This gives $|x_i| < u$ so $|s_i| \leq 2u$, and $|x_{i-1}| \leq 1-u$ so $|s_i| + |x_{i-1}| \leq 1+u$.

On the other hand, $|s_{i-1} + e_i| \geq 1 + \frac{5}{8}u$.

Being so close to equality implies (by an easy case study) $s_{i-1} = 1$, $x_{i-1} = 1-u$ and $s_i = u + e_i$.

In particular, $2u^2 | e_i$.

- First, let us consider what happens at the right of i.

We saw that $x_{i-1} < 1$, so $e_i \leq u$.

What is more, $|x_i| < u$, so $\forall i' \geq i+1, |e_{i'}| \leq u^2$.

We can have additionnal information in two cases:

- Case $e_i = u$.
  Then we saw that $s_i = 2u$ and $x_i \leq u - u^2$. So we must have $s_{i+1} \geq u$ with $x_{i+1} < u$, so $s_{i+2} \neq 0$.
  In particular, $i \leq 3$.
- Case $e_i = u - 2u^2$.
  Then we saw $s_i = 2u - 2u^2$ and $x_i \leq u - u^2$. So we must have $s_{i+1} \geq u - u^2$ with $x_{i+1} \leq u - u^2$, so either there is no more non-zero $e_{i'}$ or $i \leq 3$.

- Then, let us consider what happens at the left of i.
We saw that $x_{i-1} = 1-u$ and $x_{i-2} < 1$, so $|x_{i-2}| = 1-u$.

- Case $x_{i-2} = -1 + u$.
  Then $e_{i-1} = 0$ and $s_{i-2} = u$.
  Afterwards, if $i \geq 3$, then $|x_{i-3}| \geq \frac{1}{2}u^{-1}$ so $e_{i-2} = u$ and $s_{i-3} = x_{i-3}$.
- Case $x_{i-2} = 1 - u$.
  Then $e_{i-1} = -u$ and $s_{i-2} = 2$.
  Afterwards, s_{i-2} and all the $x_{i'}, i' \leq i-3$ are divisible by 1, so it is the case for the $e_{i'}, i' \leq i - 2$.

$\star$ Finally, let us analyze the VSEB part.
We consider i_0 such that $y_j = r_{i_0-1}$, so $\epsilon_{i_0} \neq 0$.
Let i_1 be the index of the first e_i after e_{i_0} that is non-zero.
In particular, $1 \leq i_0 < i_1$ so $i_1 \geq 2$.

We suppose WLOG that $\text{uls}(e_{i_0}) = u$.
Then $|\epsilon_{i_0}| \geq u$, and $\text{ulp}(y_j) \geq 2|\epsilon_{i_0}| \geq 2u$.

$\star$ Let us suppose that $i_1 \leq 3$ and $e_{i_1} \geq \frac{5}{8}u$.

- Case $x_{i_1-2} = -1 + u$.
Then $e_{i_1-1} = 0$ so $1 \leq i_0 < i_1 - 1$, which gives $i_1 = 3$. Thus $|y_j| = |s_0| \geq \frac{1}{2}u^{-1}$ so $\text{ulp}(y_j) \geq 1$, and clearly $|y_{j+1}| < 1$.

- Case $x_{i_1-2} = 1 - u$.
Then $e_{i-1} = -u$ and all the $e_i, i \leq i_1 - 2$ are divisible by 1. Thus we must have $\epsilon_{i_0} = -u$, so $|\epsilon_{i_0} + e_{i_1}| \leq \frac{3}{8}u$. What is more, from the previous results, $|e_{i_1+1}|, \cdots, |e_5| \leq u^2$. Thus, because we are adding at most 3 of them, $|y_{j+1}| < 2u$.

$\star$ Let us suppose that $i_1 \geq 4$ (so in particular there is no need to consider beyond r_{i_1}) and $e_{i_1} \geq \frac{5}{8}u$.
From the previous results, either $|e_{i_1}| = u - 2u^2$ and this is the last non-zero e_i, or $|e_{i_1}| \leq u - 4u^2$.
We have $|\epsilon_{i_0}| + |e_{i_1}| \leq |\epsilon_{i_0}| + (u - 2u^2) \leq (2 - 2u)|\epsilon_{i_0}|$, so $|r_{i_1-1}| \leq (1-u)\text{ulp}(y_j)$.

- Case $|e_{i_1}| = u - 2u^2$.
Then, $y_{j+1} = r_{i_1-1} < \text{ulp}(y_j)$.

- Case $|e_{i_1}| \leq u - 4u^2$.
Then we have the stronger estimate $|r_{i_1-1}| \leq (1 - 2u)\text{ulp}(y_j)$. What is more, from the previous results, $|e_{i_1+1}| \leq u^2 \leq \frac{1}{2}u\text{ulp}(y_j)$. Thus, after rounding (ties-to-even), $|r_{i_1}| \leq (1 - 2u)\text{ulp}(y_j)$, so $|y_{j+1}| < \text{ulp}(y_j)$.

$\star$ Let us suppose that $0 < e_{i_1} < \frac{5}{8}u$.
We have $|\epsilon_{i_0}| + |e_{i_1}| \leq |\epsilon_{i_0}| + \frac{5}{8}u \leq (1 + \frac{5}{8})|\epsilon_{i_0}|$, so $|r_{i_1-1}| \leq \frac{13}{16}\text{ulp}(y_j)$.

- Case $|e_{i_1}| > \frac{1}{2}u$.
Similarly to what we saw before, we deduce that $|x_{i_1-2}| < 1$, so $|x_{i_1}| < u$ so $|e_{i_1+1}|, \cdots, |e_5| \leq u^2 \leq \frac{u}{2}\text{ulp}(y_j)$. Thus, because we are adding at most 3 of them, $|y_{j+1}| \leq \frac{7}{8}\text{ulp}(y_j)$ (this part uses $p \geq 4$, and ties-to-even for $p = 4$).

- Case $|e_{i_1}| = \frac{1}{2}u$.
Then, thanks to ties-to-even, $2u \big| s_{i_1-1}$. Similarly to what we saw before, we deduce that $|x_{i_1-2}| < 1$, so $|x_{i_1}| < u$ so $|e_{i_1+1}|, \cdots, |e_5| \leq u^2 \leq \frac{u}{2}\text{ulp}(y_j)$, and we conclude like in the previous case.

- Case $\frac{1}{2}u > |e_{i_1}|$ and $|e_{i_1}| \neq \frac{1}{4}u$.
Then $|r_{i_1-1}| \leq \frac{3}{4}\text{ulp}(y_j)$. What is more, $\text{uls}(e_{i_1}) \leq \frac{1}{8}u$ so $|e_{i_1+1}|, \cdots, |e_5| \leq \frac{1}{8}u \leq \frac{1}{16}\text{ulp}(y_j)$, and we conclude like in the previous cases.

- Case $|e_{i_1}| = \frac{1}{4}u$.
Then $|r_{i_1-1}| \leq \frac{5}{8}\text{ulp}(y_j)$ What is more, $\text{uls}(e_{i_1}) \leq \frac{1}{4}u$ so $|e_{i_1+1}|, \cdots, |e_5| \leq \frac{1}{4}u \leq \frac{1}{8}\text{ulp}(y_j)$. More precisely, we can not have $|e_4| = |e_3|$ (because this can not happen for $i \geq 4$) so $|e_5| \leq \text{uls}(e_4) \leq \frac{1}{16}\text{ulp}(y_j)$. We conclude like in the previous cases. □

5.2 Number of operations

In the $Merge$, if the last two numbers to sort are x_2 and y_2, there is no need to do it them because they play symetrical roles in 2Sum. Thus this part costs only 4 tests.

In VecSum, there are sadly for each block examples where Fast2Sum can't be used, so it costs 30 flops.

From theorems [REF], Fast2Sum can be used in VSEB, so it costs 12 flops and 4 tests.

6 Product of two TW

To compute the product of two TW, we simply distribute the product and aggregate the terms ensuring P-nonoverlapping with an error as small as possible.

The algorithms presented garantee commutativity, even if it is rarely usefull in practice.

6.1 Accurate algorithm

Algorithm 12 – 3Prod$^{acc}_{3,3}(x_0, x_1, x_2, y_0, y_1, y_2)$. (46 flops & 2 tests)

Require: $\bar{x}$ and $\bar{y}$ TW ; $p \geq 6$
Ensure: $\bar{r}$ TW **and** $\left|\frac{\bar{r} - \bar{x}\bar{y}}{\bar{x}\bar{y}}\right| \leq 28u^3 + 107u^4$
$z^+_{00}, z^-_{00} \leftarrow$ 2Prod(x_0, y_0)
$z^+_{01}, z^-_{01} \leftarrow$ 2Prod(x_0, y_1)
$z^+_{10}, z^-_{10} \leftarrow$ 2Prod(x_1, y_0)
$b_0, b_1, b_2 \leftarrow$ VecSum$(z^-_{00}, z^+_{01}, z^+_{10})$
$c \leftarrow \text{RN}(b_2 + x_1y_1)$ (fma)
$z_{3,1} \leftarrow \text{RN}(z^-_{10} + x_0y_2)$ (fma)
$z_{3,2} \leftarrow \text{RN}(z^-_{01} + x_2y_0)$ (fma)
$z_3 \leftarrow \text{RN}(z_{3,1} + z_{3,2})$
$e_0, e_1, e_2, e_3, e_4 \leftarrow$ VecSum$(z^+_{00}, b_0, b_1, c, z_3)$
$r_0 \leftarrow e_0$
$r_1, r_2 \leftarrow$ VSEB(2)(e_1, e_2, e_3, e_4)

6.2 Estimates of the different terms

We suppose WLOG that $1 \le x_0, y_0 < 2$, so that $|x_1|, |y_1| < 2u$ and $|x_2|, |y_2| < 2u^2$.

Then:

$$1 \le |z_{00}^+| < 4$$
$$|z_{00}^-| \le 2u \ ; \ \text{uls}(z_{00}^-) \ge 4u^2$$

$$|z_{01}^+|, |z_{10}^+| < 4u \qquad |b_2| \le 4u^2$$
$$|x_1 y_1| < 4u^2 - 4u^3 \qquad |c| < 8u^2$$

$$|z_{01}^-|, |z_{10}^-| \le 2u^2 \qquad |x_0 y_2|, |x_2 y_0| < 4u^2$$
$$|z_{3,1}|, |z_{3,2}| \le 6u^2 \qquad |z_3| \le 12u^2$$

$$|s_3| \le 20u^2$$

6.3 Correctness

Theorem 8. *If $\bar{x}$, $\bar{y}$ TW, then $3Prod_{3,3}^{acc}(\bar{x}, \bar{y})$ TW, provided that $p \ge 6$.*

Lemma 9. *For all x, y FP, $\frac{1}{2}\text{ulp}(x) \big| RN(x+y)$.*

Proof of the lemma: If $\text{ufp}(y) \ge \frac{1}{2}\text{ufp}(x)$, then $\frac{1}{2}\text{ulp}(x) \big| x + y$ so $\frac{1}{2}\text{ulp}(x) \big| RN(x+y)$.

If $\text{ufp}(y) \le \frac{1}{4}\text{ufp}(x)$, then $|y| < \frac{1}{2}|x|$ so $|x+y| \ge \frac{1}{2}\text{ufp}(x)$ so $\frac{1}{2}\text{ulp}(x) \big| RN(x+y)$. □

Proof of the theorem: ⋆ First, let us prove that the last 2 lines are equivalent to computing:

$$r_0, r_1, r_2 = \text{VSEB}(3)(e_0, e_1, e_2, e_3, e_4)$$

- If $e_1 \neq 0$, then the fact that $e_0 = RN(e_0 + e_1)$ concludes immediately.

- If $e_1 = 0$, we easily see that we have $|s_1|, |s_2|, |s_3| < 16u \le \frac{1}{2}\text{ufp}(z_{00}^+)$ so that the next non-zero $|e_i|$ is strictly lower than $\frac{1}{2}\text{ulp}(e_0)$, which concludes.

⋆ Then, let us prove that, with this equivalent version, (r_0, r_1, r_2) is P-nonoverlapping.

We denote $a := RN(z_{01}^+ + z_{10}^+)$ and $s_3 := RN(c + z_3)$.

We want to show that $\text{VecSum}(z_{00}^+, b_0, b_1, s_3)$ is F-nonovelapping, with e_4 F-nonovelapping them too.

Thanks to Theorem 3, this would give (r_0, r_1, r_2) P-nonoverlapping.

- First, let us show that $(z_{00}^+, b_0, b_1, s_3)$ satisfies the conditions of Theorem 2.

First, we always have $\text{ufp}(z_{00}^+) \ge 1$ much larger than four times any other number computed ; and in case on they are non-zero $\text{ufp}(b_1) \le \frac{1}{2}\text{ulp}(b_0) < \frac{1}{2}\text{ufp}(b_0)$.

For the rest of the sequence, we suppose WLOG that $|x_1| \ge |y_1|$.

On one hand, we easily obtain $|s_3| \le 10\text{ulp}(x_1)$.

On the other hand lemma 9 gives $\frac{1}{2}\text{ulp}(x_1) \big| a$ with $\frac{1}{2}\text{ulp}(x_1) \le u^2 < \text{uls}(z_{00}^-)$ so $\frac{1}{2}\text{ulp}(x_1) \big| b_0, b_1$.

- Case $s_3 = 0$: we use $I = \emptyset$. (nothing more to show)
- Case $s_3 \neq 0$ and $b_0 = 0$ (so $b_1 = 0$): we use $I = \emptyset$. (idem)
- Case $s_3 \neq 0$, $b_0 \neq 0$ and $b_1 = 0$: we use $I = \{1\}$.
  Indeed, we have $\text{ufp}(s_3) \le \frac{1}{4}\text{ufp}(z_{00}^+)$, and $\text{ufp}(s_3) \le 2^{p-2}\text{uls}(b_0)$. (using $p \ge 6$)
- Case $s_3 \neq 0$, $b_0 \neq 0$ and $b_1 \neq 0$: we use $I = \{2\}$.
  Indeed, we have $\text{ufp}(s_3) \le 16\text{ufp}(b_1) \le \frac{1}{4}\text{ufp}(b_0)$ and $\text{ufp}(s_3) \le 2^{p-2}\text{uls}(b_1)$. (using $p \ge 6$)

By the way, it also allows us to call Fast2Sum instead of the three 2Sum in this part of the algorithm.

- Then, let us show that e_4 is F-nonoverlapping with the other e_i.

Firstly, $\text{ulp}(s_3) \ge 2|e_4|$.

Secondly, we saw that $\text{uls}(b_0), \text{uls}(b_1) \ge \frac{1}{2}\text{ulp}(x_1) \ge \frac{1}{20}|s_3| \ge \text{ulp}(s_3)$. (using $p \ge 6$)

Thirdly, $\text{ulp}(z_{00}^+) \ge 2u > \text{ulp}(s_3)$.

Thus e_0, e_1 and e_2 are divisible by $\text{ulp}(s_3) \ge 2|e_4|$. □

6.4 Number of operations

Before the last 3 lines, we count 22 flops.

There are 3 Fast2Sum in $\text{VecSum}(z_{00}^+, b_0, b_1, c, z_3)$, so that it costs 15 flops.

Finally, the call to VSEB costs 9 flops and 2 tests.

Thus the total is 46 flops and 2 tests.

Remark 7. *The optimisation of directly using that $e_0 = r_0$ does not save any operation, but it saves the first branching, which is very interesting.*

6.5 Bound on the error

Theorem 10. *If $\bar{x}$, $\bar{y}$ TW, then the relative error commited by $3Prod_{3,3}^{acc}(\bar{x}, \bar{y})$ is bounded by $28u^3 + 107u^4$, provided that $p \ge 6$.*

Proof: There are three sources of error: the terms that are ignored, the roundings in the computation of z_3 and c and the terms not kept in VSEB.

A naive analysis gives:

$$\begin{aligned} |\epsilon_0| &:= |x_1 y_2 + x_2 y_1 + x_2 y_2| \\ &\le 2(2u - 2u^2)(2u^2 - 2u^3) + (2u^2 - 2u^3)^2 \\ &\le 8u^3 - 11.9u^4 \end{aligned}$$

$$\begin{aligned} |\epsilon_1| &:= |(z_{10}^- + x_0 y_2) - z_{3,1}| \\ &\le u\text{ufp}(z_{10}^- + x_0 y_2) \\ &\le u\text{ufp}(2u^2 + 4u^2) \\ &\le 4u^3 \end{aligned}$$

$$\begin{aligned} |\epsilon_2| &:= |(z_{01}^- + x_2 y_0) - z_{3,2}| \\ &\le 4u^3 \end{aligned}$$

$$\begin{aligned} |\epsilon_3| &:= |(z_{3,1} + z_{3,2}) - z_3| \\ &\le u\text{ufp}(z_{3,1} + z_{3,2}) \\ &\le u\text{ufp}(6u^2 + 6u^2) \\ &\le 8u^3 \end{aligned}$$

$$\begin{aligned} |\epsilon_4| &:= |(b_2 + x_1 y_1) - c| \\ &\le u\text{ufp}(b_2 + x_1 y_1) \\ &\le u\text{ufp}(4u^2 + (4u^2 - 4u^3)) \\ &\le 4u^3 \end{aligned}$$

$$\begin{array}{rcl} |\epsilon_5| & := & |(z_{00}^+ + b_0 + b_1 + c + z_3) - (r_0 + r_1 + r_2)| \\ & \leq & (2u^3 + 4.2u^4)|z_{00}^+ + b_0 + b_1 + c + z_3| \end{array}$$

Thus

$$|\epsilon_0 + \cdots + \epsilon_4| \leq 28u^3 - 11.9u^4$$

$$|\epsilon_5| \leq (2u^3 + 4.2u^4)(\bar{x}\bar{y} + 28u^3 - 11.9u^4)$$

Yet, $\bar{x}, \bar{y} \geq 1 - (2u - 2u^2) - (2u^2 - 2u^3) \geq 1 - 2u$ so $\bar{x}\bar{y} \geq 1 - 4u$.

We get

$$\begin{array}{rcl} \left|\frac{\bar{r}-\bar{x}\bar{y}}{\bar{x}\bar{y}}\right| & \leq & (2u^3 + 4.2u^4) + \frac{(28u^3 - 11.9u^4)(1+2u^3+4.2u^4)}{1-4u} \\ & \leq & 30u^3 + 111u^4 \end{array}$$

We want to improve this bound basing on the fact that, when the other error terms are big, we must have $\epsilon_5 = 0$.

- We suppose that $|y_2| < u^2$.
Then $|x_0 y_2| \leq (2-2u)(u^2 - u^3)$, so that $|\epsilon_1| \leq 2u^3$.
We get $|\epsilon_0 + \cdots + \epsilon_4| \leq 26u^3 - 11.9u^4$ and finally

$$\begin{array}{rcl} \left|\frac{\bar{r}-\bar{x}\bar{y}}{\bar{x}\bar{y}}\right| & \leq & (2u^3 + 4.2u^4) + \frac{(26u^3 - 11.9u^4)(1+2u^3+4.2u^4)}{1-4u} \\ & \leq & 28u^3 + 103u^4 \end{array}$$

- We suppose that $|c| < 4u^2$.
Then $|\epsilon_4| \leq 2u^3$, and we conclude similarly.

- We suppose that $|z_3| < 4u^2$.
Then $|\epsilon_3| \leq 2u^3$, and we conclude similarly.

- The only case left to consider is $u \leq |x_1|, |y_1| < 2u$, $u^2 \leq |x_2|, |y_2| < 2u^2$, $|c| \geq 4u^2$ and $|z_3| \geq 4u^2$
We also have $|z_{00}^+| \geq 1$, $\mathrm{uls}(z_{00}^-) \geq 4u^2$ and $|z_{01}^+|, |z_{10}^+| \geq u$, so that z_{00}^+,b_0,b_1,c and z_3 are all divisible by $8u^3$.
What is more, $|z_{00}^+ + b_0 + b_1 + c + z_3| < 5$.
Thus we have $|r_0| \leq 5$ so a fourth non-zero term would by P-nonoverlapping be $< 8u^3$, which is excluded.
So $\epsilon_5 = 0$, which gives

$$\begin{array}{rcl} \left|\frac{\bar{r}-\bar{x}\bar{y}}{\bar{x}\bar{y}}\right| & \leq & \frac{28u^3 - 11.9u^4}{1-4u} \\ & \leq & 28u^3 + 107u^4 \end{array}$$

□

This bound is very tight.
Indeed, for instance for binary64, if we take:

$$\begin{array}{c} (x_0, x_1, x_2) = (1 + (13 \cdot 2^{26} + 28)u, 2u - 2^{27}u^2, 2u^2 - 4u^3) \\ (y_0, y_1, y_2) = (1 + 7 \cdot 2^{27}u, 2u - (2^{28} - 8)u^2, 2u^2 - 4u^3) \end{array}$$

we get a relative error of $\approx (28 - 10^{-5})u^3$.

6.6 Faster version

In this version, e_4 is not computed.

Algorithm 13 – 3Prod$_{3,3}^{fast}(x_0, x_1, x_2, y_0, y_1, y_2)$. (38 flops & 1 test)

Require: $\bar{x}$ and $\bar{y}$ TW ; $p \geq 6$
Ensure: $\bar{r}$ TW **and** $\left|\frac{\bar{r}-\bar{x}\bar{y}}{\bar{x}\bar{y}}\right| \leq 44u^3 + 176u^4$
[same 5 first lines as 12]
$z_{3,1} \leftarrow \mathrm{RN}(z_{10}^- + x_0 y_2)$ (fma)
$z_{3,2} \leftarrow \mathrm{RN}(z_{01}^- + x_2 y_0)$ (fma)
$z_3 \leftarrow \mathrm{RN}(z_{3,1} + z_{3,2})$
$s_3 \leftarrow \mathrm{RN}(c + z_3)$
$e_0, e_1, e_2, e_3 \leftarrow \mathrm{VecSum}(z_{00}^+, b_0, b_1, s_3)$
$r_0 \leftarrow e_0$
$r_1, r_2 \leftarrow \mathrm{VSEB}(2)(e_1, e_2, e_3)$

This saves 8 operations and 1 test.

Correctness is still ensured for the same reasons, provided that $p \geq 6$.

Concerning the error, we have the same $\epsilon_0, \cdots, \epsilon_4$, but we also have

$$\begin{array}{rcl} |\epsilon_4'| & := & |(c + z_3) - s_3| \\ & \leq & u\mathrm{ufp}(c + z_3) \\ & \leq & u\mathrm{ufp}(12u^2 + 8u^2) \\ & \leq & 16u^3 \end{array}$$

Similarly to the previous case, we can make as is $\epsilon_5 = 0$, and we get (provided that $p \geq 6$)

$$\begin{array}{rcl} \left|\frac{\bar{r}-\bar{x}\bar{y}}{\bar{x}\bar{y}}\right| & \leq & \frac{44u^3 - 11.9u^4}{1-4u} \\ & \leq & 44u^3 + 176u^4 \end{array}$$

This bound is very tight too, because the same input gives a relative error of $\approx (44 - 10^{-5})u^3$.

7 Product of a DW with a TW

To compute the product of a DW with a TW, we use the same algorithms as previously, slightly simplified using $x_2 = 0$.

Additionally to being interesting in itself, this operation will be used to compute the reciprocal and the quotient, which justifies a detailled analysis.

7.1 Accurate algorithm

Algorithm 14 – 3Prod$_{2,3}^{acc}(x_0, x_1, y_0, y_1, y_2)$. (45 flops & 2 tests)

Require: $\bar{x}$ DW and $\bar{y}$ TW ; $p \geq 6$
Ensure: $\bar{r}$ TW **and** $\left|\frac{\bar{r}-\bar{x}\bar{y}}{\bar{x}\bar{y}}\right| \leq 10.5u^3 + 39u^4$
[same 5 first lines as 12]
$z_{3,1} \leftarrow \mathrm{RN}(z_{10}^- + x_0 y_2)$ (fma)
$z_3 \leftarrow \mathrm{RN}(z_{3,1} + z_{01}^-)$
$e_0, e_1, e_2, e_3, e_4 \leftarrow \mathrm{VecSum}(z_{00}^+, b_0, b_1, c, z_3)$
$r_0 \leftarrow e_0$
$r_1, r_2 \leftarrow \mathrm{VSEB}(2)(e_1, e_2, e_3, e_4)$

7.2 Estimates of the different terms

We suppose WLOG that $1 \leq x_0, y_0 < 2$, so that $|x_1| \leq u$, $|y_1| < 2u$ and $|y_2| < 2u^2$.

Then:

$$1 \leq |z_{00}^+| < 4$$
$$|z_{00}^-| \leq 2u \ ; \ \text{uls}(z_{00}^-) \geq 4u^2$$
$$|z_{01}^+| < 4u \ ; \ |z_{10}^+| < 2u \qquad |b_2| \leq 4u^2$$
$$|x_1 y_1| < 2u^2 \qquad |c| \leq 6u^2$$
$$|z_{01}^-| \leq 2u^2 \ ; \ |z_{10}^-| \leq u^2 \qquad |x_0 y_2| < 4u^2$$
$$|z_{3,1}| \leq 5u^2 \qquad |z_3| \leq 7u^2$$
$$|s_3| \leq 13u^2$$

7.3 Correctness and number of operations

Given this is a particular case of the previous algorithm, correctness is directly ensured, provided that $p \geq 6$.

Compared to the previous algorithm, we saved 1 operation, so that the total is 41.

7.4 Bound on the error

Theorem 11. *If $\bar{x}$ DW and $\bar{y}$ TW, then the relative error commited by $3Prod_{2,3}^{acc}(\bar{x}, \bar{y})$ is bounded by $10.5u^3 + 39u^4$, provided that $p \geq 6$.*

Proof: The new naive error analysis gives:

$$\begin{aligned} |\epsilon_0| &:= |x_1 y_2| \\ &= 2u^3 - 2u^4 \end{aligned}$$

$$\begin{aligned} |\epsilon_1| &:= |(z_{10}^- + x_0 y_2) - z_{3,1}| \\ &\leq u\text{ufp}(u^2 + 4u^2) \\ &\leq 4u^3 \end{aligned}$$

$$\begin{aligned} |\epsilon_3| &:= |(z_{3,1} + z_{10}^-) - z_3| \\ &\leq u\text{ufp}(5u^2 + 2u^2) \\ &\leq 4u^3 \end{aligned}$$

$$\begin{aligned} |\epsilon_4| &:= |(b_2 + x_1 y_1) - c| \\ &\leq u\text{ufp}(4u^2 + 2u^2) \\ &\leq 4u^3 \end{aligned}$$

$$\begin{aligned} |\epsilon_5| &:= |(z_{00}^+ + b_0 + b_1 + c + z_3) - (r_0 + r_1 + r_2)| \\ &\leq (2u^3 + 4.2u^4)|z_{00}^+ + b_0 + b_1 + c + z_3| \end{aligned}$$

We get $|\epsilon_0 + \cdots + \epsilon_4| \leq 14u^3 - 2u^4$, and finally

$$\begin{aligned} \left|\frac{\bar{r} - \bar{x}\bar{y}}{\bar{x}\bar{y}}\right| &\leq (2u^3 + 4.2u^4) + \frac{(14u^3 - 2u^4)(1 + 2u^3 + 4.2u^4)}{1 - 4u} \\ &\leq 16u^3 + 62u^4 \end{aligned}$$

$\star$ First, we distinguish cases depending on whether $\bar{x}\bar{y}$ is a lot larger than 1.

- We suppose that $|\epsilon_1| > 2u^2$.

Then, given $|z_{10}^-| \leq u^2$, we must have $|x_0 y_2| > 3u^2$ so $|x_0| > 1.5$, so that $\bar{x}\bar{y} \geq 1.5 - 6u$.

- We suppose that $|\epsilon_4| > 2u^2$.

Then, given $|x_1 y_1| \leq 2u^2$, we must have $|b_2| > 2u^2$ so $|z_{01}^+ + z_{10}^+| > 4u$.

This gives us $x_0(2u - 2u^2) + y_0 u \geq 4u$, so that $x_0 y_0 \geq 1.5$ and $\bar{x}\bar{y} \geq 1.5 - 5u$.

Thus $\bar{x}\bar{y} \geq 1.5 - 5u$ (instead of $1 - 4u$) or $|\epsilon_1|, |\epsilon_4| \leq 2u^3$ (instead of $4u^3$ each).

$\star$ Then, we want like previously to use that having $\epsilon_5 \neq 0$ is constraining.

- We suppose that $\text{ufp}(r_0) \geq 2$.

Then by P-nonoverlapping we must have $\bar{r} \geq 2 - 4u$ so by the naive error bound $\bar{x}\bar{y} \geq 2 - 5u$.

- We suppose that $\text{ufp}(r_0) \leq 1$.

Then by P-nonoverlapping, if we want $\epsilon_5 \neq 0$, we must have a number among z_{00}^+, z_{00}^-, z_{01}^+, z_{10}^+, c and z_3 that is not divisible by $2u^3$, so in particular whose absolute value is strictly smaller than u^2.

- We saw that z_{00}^+ and z_{00}^- are always divisible by $4u^2$.
- If $|z_{01}^+| < u^2$, then $|x_1| < u^2$ so $|\epsilon_0| \leq 2u^4$ (instead of $2u^3 - 2u^4$)
- If $|z_{10}^+| < u^2$, then $|y_1| < u^2$ so $|y_2| < u^3$ so $|\epsilon_0| \leq u^4$ (instead of $2u^3 - 2u^4$)
- If $|z_3| < u^2$, then $|\epsilon_3| \leq \frac{1}{2}u^3$ (instead of $4u^3$).
- If $|c| < u^2$, then $|\epsilon_4| \leq \frac{1}{2}u^3$ (instead of $4u^3$).

$\star$ Finally, we are left with several cases:

- Case $\bar{x}\bar{y} \geq 2 - 5u$.

Then

$$\begin{aligned} \left|\frac{\bar{r} - \bar{x}\bar{y}}{\bar{x}\bar{y}}\right| &\leq (2u^3 + 4.2u^4) + \frac{(14u^3 - 2u^4)(1 + 2u^3 + 4.2u^4)}{2 - 5u} \\ &\leq 10u^3 \end{aligned}$$

- Case $2 - 5u > \bar{x}\bar{y} \geq 1.5 - 5u$ and $\epsilon_5 = 0$.

Then

$$\begin{aligned} \left|\frac{\bar{r} - \bar{x}\bar{y}}{\bar{x}\bar{y}}\right| &\leq \frac{14u^3 - 2u^4}{1.5 - 5u} \\ &\leq 10u^3 \end{aligned}$$

- Case $2 - 5u > \bar{x}\bar{y} \geq 1.5 - 5u$ and $\epsilon_5 \neq 0$.

Then

$$\begin{aligned} \left|\frac{\bar{r} - \bar{x}\bar{y}}{\bar{x}\bar{y}}\right| &\leq (2u^3 + 4.2u^4) + \frac{(12u^3 + 2u^4)(1 + 2u^3 + 4.2u^4)}{1.5 - 5u} \\ &\leq 10u^3 + 34u^4 \end{aligned}$$

- Case $1.5 - 6u > \bar{x}\bar{y}$ and $\epsilon_5 = 0$.

Then

$$\begin{aligned} \left|\frac{\bar{r} - \bar{x}\bar{y}}{\bar{x}\bar{y}}\right| &\leq \frac{10u^3 - 2u^4}{1 - 4u} \\ &\leq 10u^3 + 41u^4 \end{aligned}$$

- Case $1.5 - 6u > \bar{x}\bar{y}$ and $\epsilon_5 \neq 0$.

Then (the worst case being the one where only $|c| < u^2$, because it is partially redundant with what $1.5 - 6u > \bar{x}\bar{y}$ gives)

$$\begin{aligned} \left|\frac{\bar{r} - \bar{x}\bar{y}}{\bar{x}\bar{y}}\right| &\leq (2u^3 + 4.2u^4) + \frac{(8.5u^3 - 2u^4)(1 + 2u^3 + 4.2u^4)}{1 - 4u} \\ &\leq 10.5u^3 + 39u^4 \end{aligned}$$

□

It is unclear whether this bound is very tight, but it is not so far from optimality.

Indeed, for instance for binary64, if we take:

$$(x_0, x_1) = (1 + 3 \cdot 2^{27}u, u - 2^{27}u^2)$$
$$(y_0, y_1, y_2) = (1 + (3 \cdot 2^{26} + 6)u, 2u - 5 \cdot 2^{27}u^2, 2u^2 - 26u^3)$$

we get a relative error of $\approx (10 - 2 \cdot 10^{-6})u^3$.

7.5 Faster version

Algorithm 15 – 3Prod$_{2,3}^{fast}(x_0, x_1, y_0, y_1, y_2)$. (37 flops & 1 test)

Require: $\bar{x}$ DW and $\bar{y}$ TW ; $p \geq 6$
Ensure: $\bar{r}$ TW **and** $\left|\frac{\bar{r}-\bar{x}\bar{y}}{\bar{x}\bar{y}}\right| \leq 10.5u^3 + 39u^4$
[same 5 first lines as 12]
$z_{3,1} \leftarrow \mathrm{RN}(z_{10}^{-} + x_0 y_2)$ (fma)
$z_3 \leftarrow \mathrm{RN}(z_{3,1} + z_{01}^{-})$
$s_3 \leftarrow \mathrm{RN}(c, z_3)$
$e_0, e_1, e_2, e_3 \leftarrow \mathrm{VecSum}(z_{00}^{+}, b_0, b_1, s_3)$
$r_0 \leftarrow e_0$
$r_1, r_2 \leftarrow \mathrm{VSEB}(2)(e_1, e_2, e_3)$

We have an additionnal

$$\begin{array}{rcl} |\epsilon_4'| & := & |(c + z_3) - s_3| \\ & \leq & u\mathrm{ufp}(c + z_3) \\ & \leq & u\mathrm{ufp}(6u^2 + 7u^2) \\ & \leq & 8u^3 \end{array}$$

The case study is more profitable here because we can consider s_3 instead of c and z_3, which eliminates the case with redundancy.

We get, provided that $p \geq 6$,

$$\begin{array}{rcl} \left|\frac{\bar{r}-\bar{x}\bar{y}}{\bar{x}\bar{y}}\right| & \leq & \max\left(\frac{18u^3-2u^4}{1-4u}\ , \right. \\ & & \left. (2u^3 + 4.2u^4) + \frac{(16u^3+2u^4)(1+2u^3+4.2u^4)}{1-4u}\right) \\ & \leq & 18u^3 + 75u^4 \end{array}$$

This bound is very tight, because the same input gives a relative error of $\approx (18 - 2.4 \cdot 10^{-6})u^3$.

8 Reciprocal of a TW

To compute the reciprocal of a TW, we use an algorithm based on Newton-Raphson iteration. [REF]

This algorithm requires the stronger constraint $p \geq 9$.

8.1 Algorithm

Algorithm 16 – 3Reci(x_0, x_1, x_2). (75 flops & 2 tests, resp. 67 flops & 1 test)

Require: $\bar{x}$ TW ; $p \geq 9$
Ensure: $\bar{y}$ TW **and** $\left|\frac{\bar{y}-\bar{x}^{-1}}{\bar{x}^{-1}}\right| \leq 11.5u^3 + 1473u^4$, resp. $19u^3 + 1510u^4$
$a \leftarrow (1 + \mathrm{ulp}(1))/x_0$
$h_{0,1}, h_{1,1} \leftarrow \mathrm{2Prod}(x_0, a)$
$h_0 \leftarrow 2 - h_{0,1}$ (exact)
$h_1 \leftarrow \mathrm{RN}(-h_{1,1} - ax_1)$ (fma)
$b_{0,1}, b_{1,1} \leftarrow \mathrm{2Prod}(a, h_0)$
$b_{1,2} \leftarrow \mathrm{RN}(b_{1,1} + ah_1)$ (fma)
$\bar{b} \leftarrow \mathrm{Fast2Sum}(b_{0,1}, b_{1,2})$
$\bar{i} \leftarrow 2 - 3Prod_{2,3}(\bar{b}, \bar{x})$
$\bar{y} \leftarrow 3Prod_{2,3}(\bar{b}, \bar{i})$

8.2 Bound on the error

For now, the computation of $\bar{i}$ is seen as a black box.

The computation of h_0 is errorless thanks to Sterbenz's lemma because

$$\begin{array}{rcl} |h_{0,1}| & \geq & (1-u)|x_0 a| \\ & \geq & (1-u)|x_0|(1-u)\frac{1+2u}{|x_0|} \\ & \geq & 1 - 3u^2 > 1 - u \\ \rightarrow |h_{0,1}| & \geq & 1 \end{array}$$

This is why we used $1 + \mathrm{ulp}(1)$ instead of 1.

We have

$$\begin{array}{rcl} \left|a - \frac{1}{x_0}\right| & \leq & \left|a - \frac{1+2u}{x_0}\right| + \left|\frac{1+2u}{x_0} - \frac{1}{x_0}\right| \\ & \leq & u\left|\frac{1+2u}{x_0}\right| + \left|\frac{2u}{x_0}\right| \\ & \leq & (3u + 2u^2)\left|\frac{1}{x_0}\right| \end{array}$$

$\rightarrow \left|\frac{1-4u}{x_0}\right| \leq |a| \leq \left|\frac{1+4u}{x_0}\right|$
$\rightarrow |h_0| \leq 1 + 4u\ ;\ |h_1| \leq u|x_0 a| + |a| \cdot 2u|x_0| \leq 3u + 10u^2$

$$\begin{array}{rcl} \left|\frac{1}{x_0} - \frac{1}{x_0+x_1}\right| & = & \left|\frac{1}{x_0+x_1}\right|\left|1 - \frac{x_0+x_1}{x_0}\right| \\ & \leq & (2u - 2u^2)\left|\frac{1}{x_0+x_1}\right| \\ \left|a - \frac{1}{x_0+x_1}\right| & \leq & (5u + 4u^2)\left|\frac{1}{x_0+x_1}\right| \\ \left|a(2 - (x_0+x_1)a) - \frac{1}{x_0+x_1}\right| & = & |x_0+x_1|\left(a - \frac{1}{x_0+x_1}\right)^2 \\ & \leq & (25u^2 + 41u^3)\left|\frac{1}{x_0+x_1}\right| \end{array}$$

$\rightarrow |a| \leq \left|\frac{1+6u}{x_0+x_1}\right|$

$$\begin{array}{rcl} \left|\bar{h} - (2 - (x_0+x_1)a)\right| & = & |h_1 - (-h_{1,1} - ax_1)| \\ & \leq & u|h_{1,1} + ax_1| \\ & \leq & u(u|ax_0| + |2ux_0\frac{1+4u}{x_0}|) \\ & \leq & 3u^2 + 12u^3 \\ \left|\bar{b} - a\bar{h}\right| & = & |b_{1,2} - (b_{1,1} + ah_1)| \\ & \leq & u|b_{1,1} + ah_1| \\ & \leq & u\left(u|ah_0| + |ah_1|\right) \\ & \leq & (4u^2 + 14u^3)|a| \end{array}$$

$$\begin{aligned}\left|\bar{b}-\tfrac{1}{x_0+x_1}\right| &\leq (7u^2+26u^3)|a|+(25u^2+41u^3)\left|\tfrac{1}{x_0+x_1}\right| \\ &\leq (32u^2+110u^3)\left|\tfrac{1}{x_0+x_1}\right|\end{aligned}$$

$$\begin{aligned}\left|\tfrac{1}{x_0+x_1}-\tfrac{1}{\bar{x}}\right| &= \left|\tfrac{1}{\bar{x}}\right|\left|1-\tfrac{\bar{x}}{x_0+x_1}\right| \\ &\leq \tfrac{2u^2}{1-2u}\left|\tfrac{1}{\bar{x}}\right| \\ &\leq (2u^2+5u^3)\left|\tfrac{1}{\bar{x}}\right| \\ \left|\bar{b}-\tfrac{1}{\bar{x}}\right| &\leq (34u^2+115u^3)\left|\tfrac{1}{\bar{x}}\right| \\ \left|\bar{b}(2-\bar{x}\bar{b})-\tfrac{1}{\bar{x}}\right| &= |\bar{x}|(\bar{b}-\tfrac{1}{\bar{x}})^2 \\ &\leq 1172u^4\left|\tfrac{1}{\bar{x}}\right|\end{aligned}$$

$$\rightarrow \left|\tfrac{1-35u^2}{\bar{x}}\right| \leq |\bar{b}| \leq \left|\tfrac{1+35u^2}{\bar{x}}\right|$$

Remark 8. *This term will end to be neglectible if p is large, because the precision is doubled at each step so it jumps from 2 to 4 instead of just 3.*

Thus it is not a problem that computations were not performed precisely, for instance starting with $1+\text{ulp}(1)$ instead of 1.

We denote δ_1 the relative error made when computing $\bar{i}$ (taken relatively to $\bar{x}\bar{b}$) and δ_2 the one for $\bar{y}$. They are supposed less than $20u^3$ (see later).

$$\begin{aligned}\left|\bar{i}-(2-\bar{x}\bar{b})\right| &\leq \delta_1|\bar{x}\bar{b}| \\ &\leq \delta_1(1+35u^2) \\ |\bar{i}| &\leq |2-\bar{x}\bar{b}|+\delta_1(1+35u^2) \\ &\leq 1+71u^2 \\ \left|\bar{y}-\bar{b}\bar{i}\right| &\leq \delta_2|\bar{b}\bar{i}| \\ &\leq \delta_2(1+71u^2)|\bar{b}|\end{aligned}$$

$$\begin{aligned}\left|\bar{y}-\tfrac{1}{\bar{x}}\right| &\leq \begin{array}{c}\delta_1(1+35u^2)|\bar{b}| \\ +\delta_2(1+71u^2)|\bar{b}| \\ +1172u^4\left|\tfrac{1}{\bar{x}}\right|\end{array} \\ &\leq \begin{pmatrix}\delta_1(1+71u^2) \\ +\delta_2(1+107u^2) \\ +1172u^4\end{pmatrix}\left|\tfrac{1}{\bar{x}}\right|\end{aligned}$$

To conclude, we only have to bound δ_1 and δ_2 as precisely as possible depending on the algorithms used.

8.3 Computation of $\bar{i}$

We saw that $|\bar{b}\bar{x}-1| \leq 34u^2+74u^3$.

Inside the computation of $3Prod_{2,3}(\bar{b},\bar{x})$, we saw that $|\bar{e}-\bar{b}\bar{x}| \leq 20u^3$.

Thus $|\bar{e}-1| \leq 35u^2$.

Given $\bar{e}$ is F-nonoverlapping, we have $|e_0-\bar{e}| \leq (1-2^{-4})\text{uls}(e_0)$, so that $|e_0|>1$ would imply $|\bar{e}| \geq (1+2u)-(1-2^{-4})2u = 1+2^{-3}u > 1+35u^2$, excluded

Thus $e_0 = 1$.

Remark 9. *This property is the one that necessits a priori $p \geq 9$, even if it would probably also work for $p = 8, 7$ and maybe even 6.*

Therefore, in order to compute $2-3Prod_{2,3}(\bar{b},\bar{x})$, we can simply replace $e_1,\ldots$ by their opposites by turning some $+$ into $-$ operations and conversly in order not to waste any operations.

Correctness and the error bound are still ensured for the same reasons.

For instance, if we choose to use the accurate version, we obtain algorithm 17.

One operation (the computation of e_0) is saved compared to the general version.

Algorithm 17 – 2 - 3Prod$^{acc}_{2,3}(b_0,b_1,x_0,x_1,x_2)$. (44 flops & 2 tests)

$z^+_{00}, z^-_{00} \leftarrow$ 2Prod(b_0,x_0)
$z^+_{01}, z^-_{01} \leftarrow$ 2Prod(b_0,x_1)
$z^+_{10}, z^-_{10} \leftarrow$ 2Prod(b_1,x_0)
$b'_0, b'_1, b'_2 \leftarrow$ VecSum$(z^-_{00}, z^+_{01}, z^+_{10})$
$c \leftarrow$ RN$(b'_2 + b_1x_1)$ (fma)
$z_{3,1} \leftarrow$ RN$(z^-_{10} + b_0x_1)$ (fma)
$z_3 \leftarrow$ RN$(z_{3,2} + z^-_{01})$
$s_1, e_2, e_3, e_4 \leftarrow$ VecSum$(-b'_0, -b'_1, -c, -z_3)$
$(i_0 = 1)$
$e_1 \leftarrow$ Fast2Sum$_2(1)(-z^+_{00}, s_1)$
$i_1, i_2 \leftarrow$ VSEB$(2)(e_1, e_2, e_3, e_4)$

8.4 Computation of $\bar{y}$

We have very precise information about $\bar{i}$, so we use a modified version of the fast algorithm.

Algorithm 18 – 3Prod$^{fast}_{2,3}(b_0,b_1,(1),i_1,i_2)$. (20 flops)

$z^+_{01}, z^-_{01} \leftarrow$ 2Prod(b_0,i_1)
$b'_0, b'_1 \leftarrow$ Fast2Sum(b_1, z^+_{01})
$z_{3,1} \leftarrow$ RN$(z^-_{01} + b_1i_1)$ (fma)
$z_3 \leftarrow$ RN$(z_{3,1} + b_0i_2)$ (fma)
$s_3 \leftarrow$ RN$(b'_1 + z_3)$
$e_0, e_1, e_2 \leftarrow$ VecSum(b_0, b'_0, s_3)
$y_0 \leftarrow e_0$
$y_1, y_2 \leftarrow$ Fast2Sum(e_1, e_2)

Remark 10. *In VecSum(b_0, b'_0, s_3), the sum of b'_0 and s_3 is performed with a 2Sum in order to ensure correctness.*

On the contrary, a Fast2Sum can be used for the sum of b_1 and z^+_{01} because if the condition to be errorless is false, it means that b_1 is very small, so that the global error will be small anyway.

Given the estimates on $\bar{i}$, basically all errors are neglectible, except the one when computing s_3.

We get:

$$\left|\frac{\bar{y}-\bar{b}\bar{i}}{\bar{b}\bar{i}}\right| \leq \frac{u^3+256u^4}{(1-2u)(1-35u^2)}$$

Thus we can take $\delta_2 = u^3+260u^4$.

8.5 Final error bound and number of operations

Before the last 2 lines, 11 operations are used.

If we use the accurate version for the first $3Prod_{2,3}$, then we can use $\delta_1 = 10.5u^3+39u^4$ and finally

Theorem 12. *If $\bar{x}$ TW, then the relative error commited by $3Reci^{acc}(\bar{x})$ is bounded by $11.5u^3 + 1473u^4$, provided that $p \geq 9$.*

The total cost is 75 flops & 2 tests.

If we use the fast version, then we can use $\delta_1 = 18u^3 + 75u^4$ and finally

Theorem 13. *If $\bar{x}$ TW, then the relative error commited by $3Reci^{fast}(\bar{x})$ is bounded by $19u^3 + 1510u^4$, provided that $p \geq 9$.*

The total cost is 67 flops & 1 tests.

9 Quotient of two TW

In this part, we also are assuming $p \geq 9$.

In order to compute $\frac{\bar{z}}{\bar{x}}$, we could simply compute the product of $\bar{z}$ and the reciprocal of $\bar{x}$.

However, it would mean that we compute something like

$$\bar{z} \times (\bar{b} \times (2 - \bar{b}\bar{x}))$$

It is a lot more advantageous to compute

$$(\bar{z}\bar{b}) \times (2 - \bar{b}\bar{x})$$

Indeed, it allows to parallelize the computations of $\bar{z}\bar{b}$ and $2 - \bar{b}\bar{x}$, and the huge optimisations allowed by the constraints on $\bar{i}$ is more efficiently used when multiplied by a TW rather than a DW.

Algorithm 19 – 3Div$(z_0, z_1, z_2, x_0, x_1, x_2)$**.** (121 flops & 4 tests, resp. 105 flops & 2 test)

Require: $\bar{z}$,$\bar{x}$ TW ; $p \geq 9$
Ensure: $\bar{y}$ TW **and** $\left|\frac{\bar{y} - \bar{z}/\bar{x}}{\bar{z}/\bar{x}}\right| \leq 24u^3 + 1519u^4$, resp. $39u^3 + 1594u^4$
[same 6 first lines as 16]
$\bar{b} \leftarrow$ Fast2Sum$(b_{0,1}, b_{1,2})$
$\bar{i} \leftarrow 2 - 3Prod_{2,3}(\bar{b}, \bar{x})$
$\bar{a} \leftarrow 3Prod_{2,3}(\bar{b}, \bar{z})$
$\bar{y} \leftarrow 3Prod_{3,3}(\bar{a}, \bar{i})$

For the computation of $2 - \bar{b}\bar{x}$, the same algorithm and error bound as previoulsy are used.

For the computation of $\bar{z}\bar{b}$, $3Prod_{2,3}$ is used (the relative error is denoted by δ_3).

For the computation of the final product, algorithm 20 is used.

Algorithm 20 – 3Prod$^{fast}_{3,3}(a_0, a_1, a_2, (1), i_1, i_2)$**.** (21 flops)

$z^+_{01}, z^-_{01} \leftarrow$ 2Prod(a_0, i_1)
$b'_0, b'_1 \leftarrow$ Fast2Sum(a_1, z^+_{01})
$z_{3,1} \leftarrow$ RN$(z^-_{01} + a_1 i_1)$ (fma)
$z_{3,2} \leftarrow$ RN$(z_{3,1} + a_0 i_2)$ (fma)
$z_3 \leftarrow$ RN$(z_{3,2} + b'_1)$
$s_3 \leftarrow$ RN$(z_3 + a_2)$
$e_0, e_1, e_2 \leftarrow$ VecSum(b_0, b'_0, s_3)
$y_0 \leftarrow e_0$
$y_1, y_2 \leftarrow$ Fast2Sum(e_1, e_2)

The error analysis is similar to the one of 18 with an additionnal $2u^3$, and we get

$$\begin{array}{rcl} \left|\frac{\bar{y} - \bar{a}\bar{i}}{\bar{a}\bar{i}}\right| & \leq & \frac{3u^3 + 256u^4}{(1-2u)(1-35u^2)} \\ & \leq & 3u^3 + 264u^4 \end{array}$$

Globally, we have

$$\left|\bar{y} - \frac{\bar{z}}{\bar{x}}\right| \leq \left(\begin{array}{c} \delta_1(1 + 71u^2) \\ +(\delta_2 + \delta_3)(1 + 107u^2) \\ +1172u^4 \end{array} \right) \left|\frac{\bar{z}}{\bar{x}}\right|$$

Concerning the operations, there are:

- 11 flops to compute $\bar{b}$
- 44 flops & 2 tests (or 36 & 1 if fast) to compute $\bar{i}$
- 45 flops & 2 tests (or 37 & 1 if fast) operations to compute $\bar{a}$
- 21 flops to compute $\bar{y}$

If we use the accurate versions, then we can use $\delta_1 = 10.5u^3 + 39u^4 = \delta_3$ and finally

Theorem 14. *If $\bar{x}$,$\bar{z}$ TW, then the relative error commited by $3Div^{acc}(\bar{z}, \bar{x})$ is bounded by $24u^3 + 1519u^4$, provided that $p \geq 9$.*

The total cost is 121 flops & 4 tests.

If we use the fast versions, then we can use $\delta_1 = 18u^3 + 75u^4 = \delta_3$ and finally

Theorem 15. *If $\bar{x}$,$\bar{z}$ TW, then the relative error commited by $3Div^{fast}(\bar{z}, \bar{x})$ is bounded by $39u^3 + 1594u^4$, provided that $p \geq 9$.*

The total number of operations is 105 flops & 2 tests.

Conclusion

PLACE PHOTO HERE

Nicolas Fabiano was born in Paris, France, in 1998. After a baccalaureat with "very good" mention, he spent 2 year in the Ecole preparatoire Louis Le Grand, and he is currently studying Computer Science at the ENS Paris. He is involved in different math associations, especially the Tournoi Français des Jeunes Mathématiciennnes et Mathématiciens (TFJM^2).

Jean-Michel Muller was born in Grenoble, France, in 1961. He received his Ph.D. degree in 1985 from the Institut National Polytechnique de Grenoble. He is Directeur de Recherches (senior researcher) at CNRS, France, and he is the co-head of GDR-IM. His research interests are in Computer Arithmetic. Dr. Muller was co-program chair of the 13th IEEE Symposium on Computer Arithmetic (Asilomar, USA, June 1997), general chair of SCAN'97 (Lyon, France, sept. 1997), general chair of the 14th IEEE Symposium on Computer Arithmetic (Adelaide, Australia, April 1999), general chair of the 22nd IEEE Symposium on Computer Arithmetic (Lyon, France, June 2015). He is the author of several books, including "Elementary Functions, Algorithms and Implementation" (3rd edition, Birkhauser, 2016), and he coordinated the writing of the "Handbook of Floating-Point Arithmetic (2nd edition Birkhäuser, 2018). He is an associate editor of the IEEE Transactions on Computers, and a fellow of the IEEE.